

Spring Boot 安全性基本身份驗證

在本 Spring 安全教程中，我們將學習如何使用 Spring 安全性提供的內置基本身份驗證來保護 REST API。

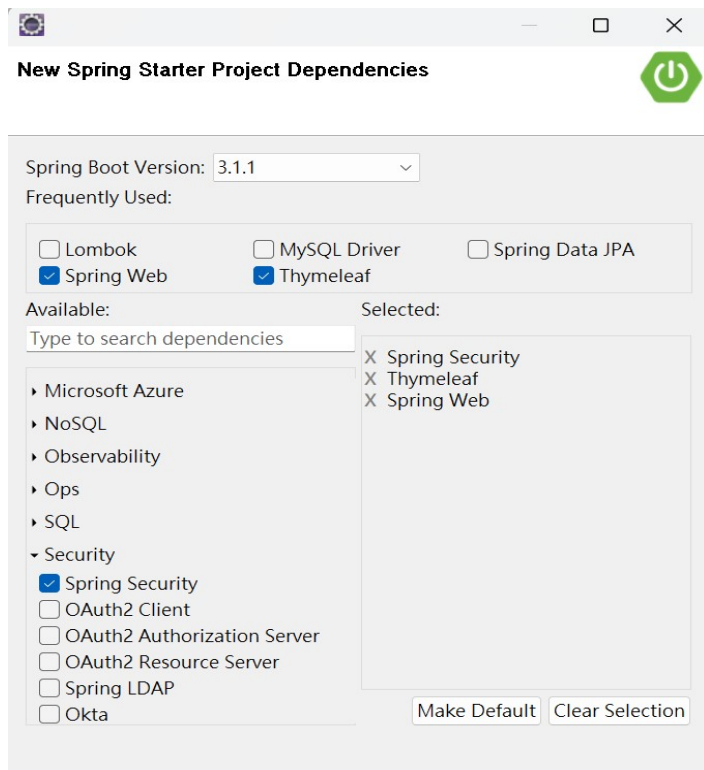
基本身份驗證概述

- 基本身份驗證是保護 REST API 的最基本選項。
- 基本身份驗證使用 **HTTP 標頭**，以便在向伺服器發出請求時提供使用者名和密碼。
- 基本身份驗證在標頭中使用 Base 64 編碼的使用者名和密碼。
- 基本身份驗證不使用 cookie，因此沒有 Session 或 Log 使用者的概念，這意味著每個請求都必須攜帶該標頭才能進行身份驗證。

Maven Dependency

為了在 Spring Boot 專案中使用 Spring 安全性，我們需要添加以下 Maven 依賴項：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```



Spring Boot 安全配置

接下來，讓我們將 Spring 安全性配置為使用基本的記憶體中身份驗證。讓我們建 *SpringSecurityConfig* 類並向其添加以下程式碼：

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.util.matcher.AntPathRequestMatcher;
```

```

@Configuration
public class SpringSecurityConfig {

    @Bean
    public static PasswordEncoder passwordEncoder(){
        return new BCryptPasswordEncoder();
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws
    Exception {

        http.csrf().disable()
            .authorizeHttpRequests((authorize) -> {
                authorize.anyRequest().authenticated();
            }).httpBasic(Customizer.withDefaults());

//        http.csrf().disable()
//            .authorizeHttpRequests((authorize) -> {
//                authorize.anyRequest().authenticated();
//            }).formLogin(Customizer.withDefaults());

//        http.csrf().disable()
//            .authorizeHttpRequests((authorize) ->
//                authorize.anyRequest().authenticated()
//            ).formLogin(
//                form -> form
//                    .loginPage("/login")
//                    .loginProcessingUrl("/login")
//                    .defaultSuccessUrl("/welcome")
//                    .permitAll()
//            ).logout(
//                logout -> logout
//                    .logoutRequestMatcher(new
    AntPathRequestMatcher("/logout"))
//                    .permitAll()
//            );

```

```

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService(){
        UserDetails mary = User.builder()
            .username("mary")
            .password(passwordEncoder().encode("password"))
            .roles("USER")
            .build();

        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder().encode("admin"))
            .roles("ADMIN")
            .build();

        return new InMemoryUserDetailsManager(mary, admin);
    }
}

```

默認情況下，Spring 安全性同時啟用基於表單和 HTTP 基本身份驗證。這裡我們使用 `httpBasic()` 元素在 *SecurityFilterChain* bean 中只定義基本身份驗證：

```

@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

    http.csrf().disable()
        .authorizeHttpRequests((authorize) -> {
            authorize.anyRequest().authenticated();
        }).httpBasic(Customizer.withDefaults());
    return http.build();
}

```

在下面的 *InMemoryUserDetailsManager* Java Configuration 中，我們建了兩個使用者並將它們存儲在 *InMemoryUserDetailsManager* 類物件中。

```

@Bean
public UserDetailsService userDetailsService(){
    UserDetails mary = User.builder()
        .username("mary")
        .password(passwordEncoder().encode("password"))
        .roles("USER")
        .build();

    UserDetails admin = User.builder()
        .username("admin")
        .password(passwordEncoder().encode("admin"))
        .roles("ADMIN")
        .build();

    return new InMemoryUserDetailsManager(mary, admin);
}

```

Spring Security 的 *InMemoryUserDetailsManager* 實現

了 *UserDetailsService*，以支援存儲在記憶體中的基於使用者名/密碼的身份驗證。

請注意，我們使用密碼 *PasswordEncoder* 進行編碼。Spring Security 的 *PasswordEncoder* 介面用於執行密碼的單向轉換，以安全地存儲密碼。我們使用 *BCryptPasswordEncoder* 類別來實作 *PasswordEncoder* 介面。

BCryptPasswordEncoder 類別實現使用廣泛支援的 **bcrypt** 演算法對密碼進行 hash 處理。

```

@Bean
public static PasswordEncoder passwordEncoder(){
    return new BCryptPasswordEncoder();
}

.password(passwordEncoder().encode("password"))

```

建立 REST API

為了測試上述 **Spring** 安全配置，讓我們建一個簡單的 **REST API** 並使用 **Spring** 安全性對其進行保護。好吧，如果我們將 **Spring** 安全依賴項添加到 **Spring** 啟動專案中，那麼預設情況下 **Spring** 安全性會保護所有應用程式 **URL**。

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

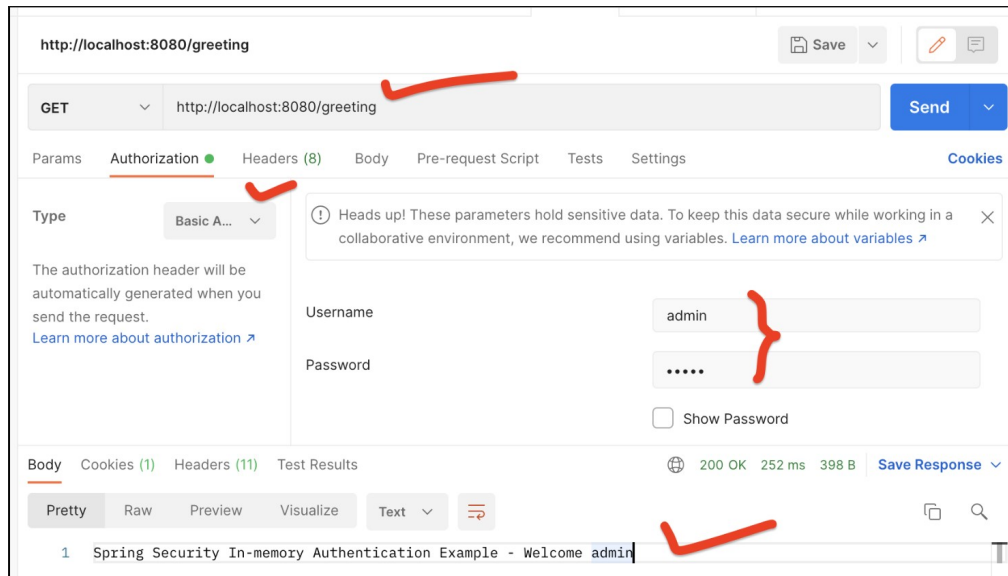
@Controller
public class WelcomeController {

    @GetMapping("/welcome")
    public String greeting() {
        return "welcome";
    }

    @GetMapping("/login")
    public String login() {
        return "login";
    }
}
```

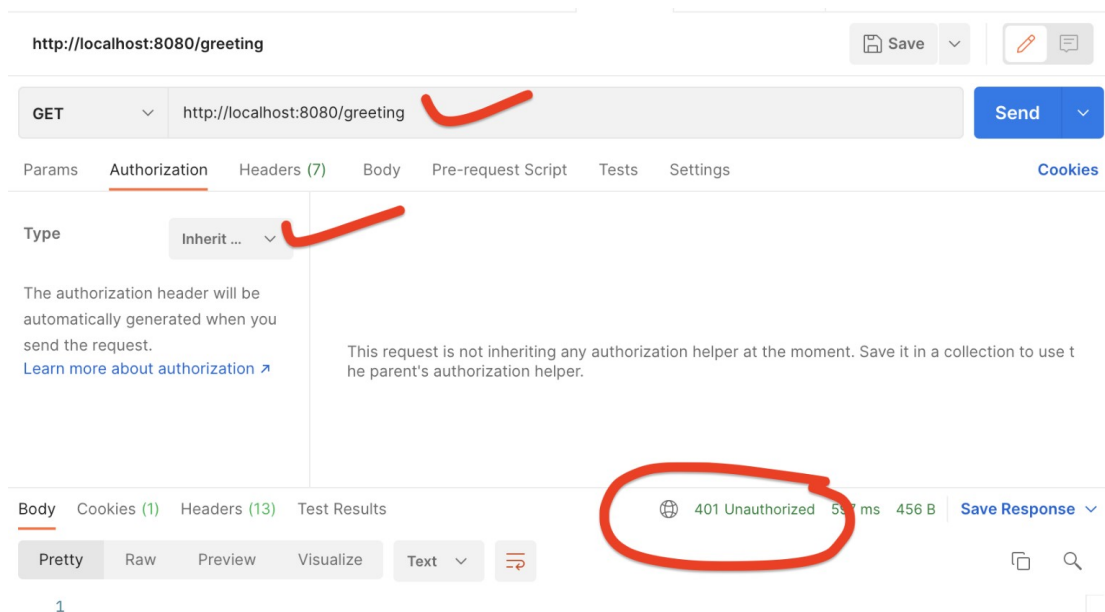
使用 **Postman** 測試 **REST API**

為了測試 **REST API**，我們必須在標頭中傳遞使用者名和密碼，這稱為基本身份驗證。



請注意，我們以管理員/管理員身份傳遞使用者名和密碼：

如果我們不傳遞使用者名和密碼，我們將獲得 401 狀態。這就是 Spring Security 保護 REST API 的方式。



結論

在本教程中，我們已經瞭解了如何配置 Spring Security 的內置 HTTP 基本身份驗證來保護 REST API。

以表單登入畫面

template/login.html

```
<!DOCTYPE html>

<html lang="en"
      xmlns:th="http://www.thymeleaf.org"
>

<head>
  <meta charset="UTF-8">
  <title>Login System</title>
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
    rel="stylesheet"

    integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65Vohh
    puuCOMmLASjC"
    crossorigin="anonymous">
</head>

<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" th:href="@{/index}">Spring Security Custom Login
    Example</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
    data-bs-target="#navbarSupportedContent"
    aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle
    navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
  </div>
```



```
</nav>

<br /><br />

<div class="container">
  <div class="row">
    <div class="col-md-6 offset-md-3">

      <div th:if="{param.error}">
        <div class="alert alert-danger">Invalid Email or Password</div>
      </div>

      <div th:if="{param.logout}">
        <div class="alert alert-success"> You have been logged out.</div>
      </div>

      <div class="card">
        <div class="card-header">
          <h2 class="text-center">Login Form</h2>
        </div>
        <div class="card-body">
          <form
            method="post"
            role="form"
            th:action="@{/login}"
            class="form-horizontal"
          >
            <div class="form-group mb-3">
              <label class="control-label"> Email</label>
              <input
                type="text"
                id="username"
```

```

        name="username"
        class="form-control"
        placeholder="Enter email address"
    />
</div>

<div class="form-group mb-3">
    <label class="control-label"> Password</label>
    <input
        type="password"
        id="password"
        name="password"
        class="form-control"
        placeholder="Enter password"
    />
</div>

<div class="form-group mb-3">
    <button type="submit" class="btn btn-primary"
>Submit</button>

    <span> Not registered ?
        <a th:href="@{/register}">Register/Signup
here</a>

    </span>
</div>
</form>
</div>
</div>
</div>
</div>

```

```
</div>
</body>
</html>
```

template/welcome.html

```
<!DOCTYPE html>
<html lang="en"
      xmlns:th="http://www.thymeleaf.org"
>
<head>
  <meta charset="UTF-8">
  <title>Registration and Login System</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
      rel="stylesheet"

integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65Vohh
puuCOMmLASjC"
      crossorigin="anonymous">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" th:href="@{/index}">Spring Security Custom Login
Example</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarSupportedContent"
aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle
navigation">
```

```

        <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarSupportedContent">
        <ul class="navbar-nav me-auto mb-2 mb-lg-0">
            <li class="nav-item">
                <a class="nav-link active" aria-current="page"
th:href="@{/logout}">Logout</a>
            </li>
        </ul>
    </div>
</div>
</nav>
<br /><br />

<body>
<div class="container">
    <div class="row">
        <h1> Welcome to Spring Security world!</h1>
    </div>
</div>

</body>
</html>

```